Software Testing and Quality Assurance

Assignment - Answers to Questions 1 to 21

Contents

Q1. What is Software Testing? Explain its objectives and importance.
Q2. What is Software Quality Assurance (SQA)? Explain its role in software development.
Q3. Differentiate between Software Testing and Quality Assurance.
Q4. Explain the major objectives of Software Quality Assurance.
Q5. What is the difference between Quality Assurance (QA) and Quality Control (QC)?
Q6. Explain the role and responsibilities of a Software Quality Assurance Engineer.
Q7. Explain the relationship between Software Testing, Quality Assurance, and Quality Control.
Q8. What are the different activities performed during Software Quality Assurance?
Q9. Explain how Software Testing helps in improving software quality.
Q10. What is a Quality Management System (QMS)? Explain its importance in software development.
Q11. Explain the importance of standards, processes, and documentation in Software Quality Assurance.
Q12. What is a Software Quality Assurance Plan (SQAP)? Explain its major components.
Q13. Explain the difference between Verification and Validation and discuss their role in software...
Q14. Explain how defects can affect the overall quality of software.
Q15-16. A banking application frequently crashes during online transactions. As a QA professional,...
Q17. Explain the importance of continuous improvement in Software Quality Assurance.
Q18. What are the major challenges faced while implementing Software Quality Assurance in an...
Q19. Explain the role of reviews, inspections, and audits in Software Quality Assurance.
Q20. Explain how QA activities are performed throughout the Software Development Life Cycle (SDLC).
Q21. Explain with a suitable real-world example how effective Software Testing and Quality Assurance...

Q1. What is Software Testing? Explain its objectives and importance.

Software testing is the process of executing a software product (or examining its work products) with the intention of finding defects and checking whether it behaves as specified. It evaluates the software against requirements, compares actual results with expected results, and gives stakeholders information about the quality and risk of the product. Testing includes both *dynamic* activities (running the code) and *static* activities (reviewing requirements, designs and code).

Objectives of Software Testing

- **Detect defects early** so that they can be fixed when they are cheapest to fix.
- **Verify requirements** - confirm that every specified function and non-functional need is implemented correctly.
- **Validate fitness for use** - confirm the product actually solves the user's problem.
- **Prevent defects** - test design and requirement reviews expose ambiguities before code is written.
- **Build confidence** in the product's reliability, security, performance and usability.
- **Provide information** to managers and customers so they can take release decisions based on facts and risk.
- **Ensure compliance** with contractual, legal, regulatory and industry standards.
- **Reduce the risk** of failure in production, where failures are most expensive.

Importance of Software Testing

- **Cost saving:** a defect found in production can cost many times more to fix than one found during requirements or design.
- **Reliability and safety:** in banking, healthcare, aviation and automotive software, failures can cause financial loss or even loss of life.
- **Security:** testing uncovers vulnerabilities before attackers do and protects user data.
- **Customer satisfaction and reputation:** stable, correct software retains users; buggy software loses them quickly.
- **Business continuity:** prevents downtime, data loss and revenue loss.
- **Better product quality:** repeated test-fix cycles steadily improve functionality, performance and usability.

Q2. What is Software Quality Assurance (SQA)? Explain its role in software development.

Software Quality Assurance (SQA) is a planned and systematic set of activities that ensures software processes and the resulting products conform to defined requirements, standards and procedures. SQA is **process-oriented and preventive**: instead of only finding defects after they appear, it aims to build quality into the way software is developed so that defects are not introduced in the first place.

Role of SQA in Software Development

- **Defines and maintains processes:** establishes development, testing, review and release procedures that the whole team follows.

- **Sets standards:** coding standards, documentation templates, naming conventions and quality criteria.
- **Monitors compliance:** checks, through reviews and audits, that the team actually follows the agreed processes.
- **Oversees testing:** ensures test plans, test cases and exit criteria are adequate and executed properly.
- **Manages defects and risks:** tracks defects, analyses root causes and highlights project risks early.
- **Measures quality:** collects metrics such as defect density, test coverage and review effectiveness.
- **Provides visibility to management:** gives an independent view of project and product quality.
- **Drives improvement:** uses lessons learned and metrics to improve the process on future projects.

Q3. Differentiate between Software Testing and Quality Assurance.

Basis	Software Testing	Quality Assurance (QA)
Definition	Executing the software to find defects and verify behaviour against requirements.	A set of planned activities that ensures the development process produces quality software.
Orientation	Product-oriented.	Process-oriented.
Nature	Corrective / detective - finds defects that already exist.	Preventive - stops defects from being introduced.
Goal	Identify bugs and confirm the product meets requirements.	Improve processes and ensure standards are followed.
Scope	A phase/activity within the SDLC.	Spans the entire SDLC and the whole organisation.
Timing	Mainly after (or while) the code is built.	From the very start of the project, before and during development.
Performed by	Testers / test engineers.	QA engineers, process owners, quality managers.
Typical activities	Test design, test execution, defect reporting, regression.	Process definition, standards, reviews, audits, training, metrics.
Example	Running login test cases and logging a bug when valid credentials fail.	Defining a rule that every requirement must be peer-reviewed before development starts.

In short: testing asks *"Does this product work correctly?"* while QA asks *"Are we building the product in the right way?"*

Q4. Explain the major objectives of Software Quality Assurance.

- **Ensure conformance to requirements:** the final product must meet functional and non-functional requirements.
- **Prevent defects:** detect weaknesses in the process and fix them so that defects are not injected.
- **Establish and enforce standards:** define coding, documentation, testing and review standards and ensure they are followed.

- **Improve the development process:** use feedback and metrics for continuous process improvement.
- **Ensure visibility and transparency:** give management objective information about quality status and risks.
- **Ensure compliance:** meet legal, regulatory, customer and organisational obligations (e.g. ISO 9001, CMMI).
- **Reduce cost of quality:** lower rework, maintenance and warranty costs by finding problems early.
- **Manage risk:** identify technical and project risks early and track them to closure.
- **Ensure that deviations are resolved:** non-conformances found in reviews or audits are documented, assigned and closed.
- **Achieve customer satisfaction:** deliver a reliable, usable product on time and within budget.

Q5. What is the difference between Quality Assurance (QA) and Quality Control (QC)?

Quality Assurance focuses on *how* software is built - the processes and standards used. **Quality Control** focuses on *what* was built - examining the actual product to find defects.

Basis	Quality Assurance (QA)	Quality Control (QC)
Focus	Process of building the product.	The product itself.
Approach	Proactive / preventive.	Reactive / detective.
Aim	Prevent defects from occurring.	Identify and correct defects that exist.
Scope	Whole SDLC, organisation-wide.	Specific product or deliverable.
Activities	Process definition, audits, training, standards, reviews of process.	Testing, inspections of the product, defect detection.
Performed by	QA team, process specialists.	QC team, testers.
Timing	Throughout development, before the product exists.	After a deliverable (code, build) is produced.
Nature	Management-level activity.	Operational / execution-level activity.
Example	Introducing a mandatory code review checklist.	Executing test cases on a release build.

Q6. Explain the role and responsibilities of a Software Quality Assurance Engineer.

A **Software Quality Assurance Engineer** ensures that the software, and the process used to create it, meet the defined quality criteria. The role combines process knowledge, technical skill and communication.

Key Responsibilities

- Understand requirements and review them for clarity, completeness, testability and consistency.
- Contribute to the SQA plan and the test strategy, including scope, resources, schedule and risks.

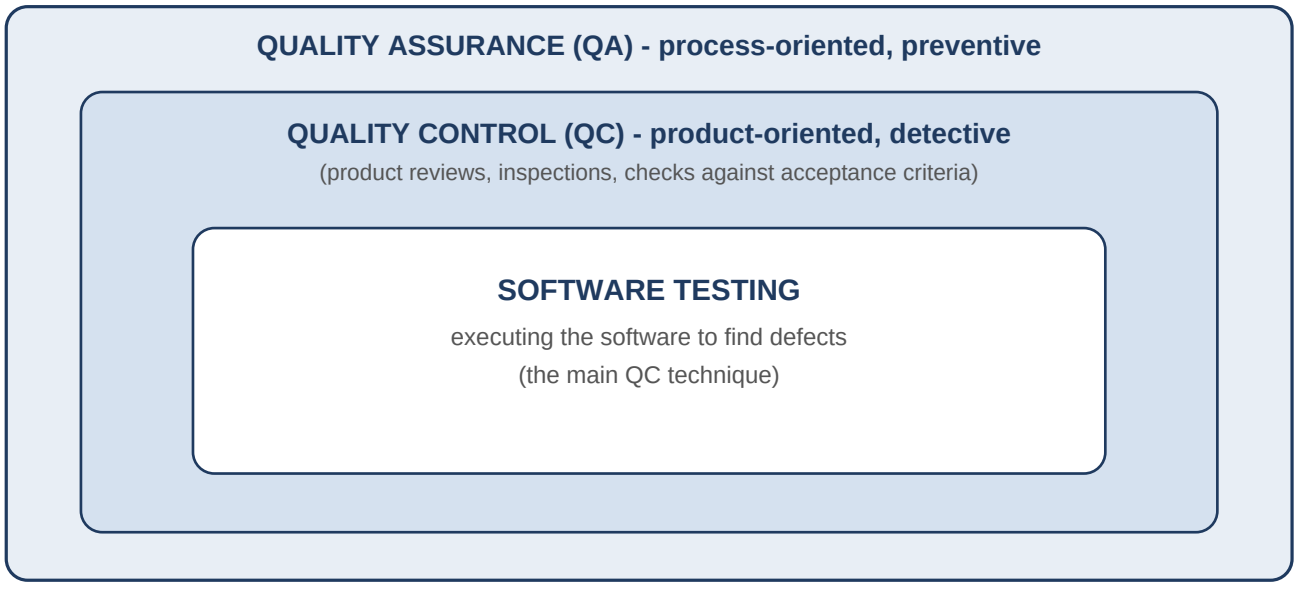
- Define, document and maintain QA processes, templates, checklists and standards.
- Design and review test cases, test data and test scenarios; verify traceability to requirements.
- Execute manual and automated tests (functional, regression, integration, performance, security, usability).
- Log defects with clear steps to reproduce, severity, priority and evidence; verify fixes and retest.
- Participate in reviews, walkthroughs and inspections of requirements, design and code.
- Conduct or support process audits and configuration audits; report non-conformances.
- Collect and analyse quality metrics (defect density, test coverage, defect leakage, pass rate).
- Perform root cause analysis of major defects and recommend preventive actions.
- Maintain and improve test automation frameworks and integrate tests in CI/CD pipelines.
- Communicate quality status, risks and release readiness to developers, managers and customers.
- Train team members on quality practices and promote a quality culture.

Skills Required

Analytical and problem-solving ability, attention to detail, knowledge of SDLC/STLC and testing techniques, familiarity with tools (defect trackers, automation tools, version control), basic scripting/SQL, and good written and verbal communication.

Q7. Explain the relationship between Software Testing, Quality Assurance, and Quality Control.

Software Testing, Quality Control and Quality Assurance are closely related and form a nested hierarchy:



- **QA is the umbrella:** it covers the entire set of planned activities that assure quality - process definition, standards, training, reviews, audits and metrics - and also includes QC.
- **QC is a subset of QA:** it checks the product against requirements to detect defects. QC results feed back to QA, which uses them to improve the process.

- **Testing is the main QC activity:** it executes the software to identify defects. Other QC activities include inspections and product reviews.

Relationship in practice: QA defines *how* testing must be done (test strategy, templates, entry/exit criteria); testers carry out the tests (QC); the defects and metrics found are analysed by QA to refine the process, so that the same defects are prevented in future. Hence, QA prevents, QC detects, and testing is the technique through which most detection happens.

Q8. What are the different activities performed during Software Quality Assurance?

1. **SQA planning:** preparing the SQA Plan - scope, standards, resources, schedule, risks and responsibilities.
2. **Defining standards and procedures:** coding standards, documentation formats, naming, review checklists.
3. **Process definition and tailoring:** selecting the lifecycle model and adapting organisational processes to the project.
4. **Reviews, walkthroughs and inspections:** examining requirements, design, code and test artefacts for defects.
5. **Testing oversight:** verifying that test plans, test cases and execution meet the strategy and coverage goals.
6. **Process and product audits:** independent checks of compliance with plans and standards.
7. **Configuration and change management:** version control, baselines, controlled changes and release management.
8. **Defect management:** logging, tracking, prioritising and analysing defects; root cause analysis.
9. **Metrics collection and analysis:** defect density, review effectiveness, test coverage, schedule and effort variance.
10. **Risk management:** identifying, assessing and monitoring quality-related project risks.
11. **Supplier / vendor quality control:** assessing third-party components and outsourced work.
12. **Training and mentoring:** making sure team members know the processes, tools and standards.
13. **Record keeping and reporting:** maintaining quality records and reporting status to management.
14. **Process improvement:** acting on lessons learned, retrospectives and audit findings.

Q9. Explain how Software Testing helps in improving software quality.

- **Finds and removes defects:** every defect fixed before release directly raises product quality.
- **Verifies requirements:** traceability between requirements and test cases ensures nothing is missed or built wrongly.
- **Improves non-functional quality:** performance, load, security, usability, compatibility and reliability testing improve the quality attributes defined in ISO/IEC 25010.

- **Early testing (shift-left):** reviewing requirements and designing tests early catches problems before they are coded.
- **Regression testing:** protects existing functionality when new features or fixes are added.
- **Reduces risk:** risk-based testing focuses effort on the areas where failure would hurt most.
- **Provides feedback to developers:** defect trends reveal weak modules and common coding mistakes.
- **Supports process improvement:** root cause analysis of escaped defects improves development practices.
- **Builds confidence:** test results and coverage give objective evidence that the software is ready for release.
- **Enables fast delivery:** automated tests in CI/CD give quick feedback, allowing frequent and safe releases.

Testing alone cannot prove that software has no defects, but systematic, well-planned testing greatly reduces the probability of failures reaching users and gives measurable evidence of quality.

Q10. What is a Quality Management System (QMS)? Explain its importance in software development.

A **Quality Management System (QMS)** is a formalised system of policies, processes, procedures, responsibilities and resources that an organisation uses to define, achieve and continually improve quality. A well-known QMS standard is **ISO 9001**; for software, ISO/IEC/IEEE 90003 gives guidance for applying it.

Main Elements of a QMS

- Quality policy and measurable quality objectives.
- Documented processes, procedures, work instructions and templates.
- Defined roles, responsibilities and authority.
- Resource management (people, tools, infrastructure, training).
- Monitoring, measurement and analysis (metrics, customer feedback).
- Internal audits and management reviews.
- Corrective and preventive action and continual improvement.

Importance in Software Development

- **Consistency:** projects follow the same proven processes, giving predictable results.
- **Customer confidence:** a certified QMS shows customers that quality is managed systematically.
- **Fewer defects and less rework** because processes include verification at each stage.
- **Clear accountability:** everyone knows their responsibilities.
- **Data-driven decisions** using metrics and audit results.
- **Regulatory and contractual compliance,** which is essential in sectors like banking and healthcare.

- **Continuous improvement** through corrective actions and management review.
- **Knowledge retention:** documented practices survive staff changes.

Q11. Explain the importance of standards, processes, and documentation in Software Quality Assurance.

Standards

Standards are agreed rules or models that define what good practice looks like. Examples: ISO 9001 (quality management), ISO/IEC 25010 (product quality model), ISO/IEC/IEEE 29119 (software testing), IEEE 730 (SQA plans), CMMI (process maturity), and organisational coding standards.

- Provide a common benchmark against which quality is measured.
- Promote consistency, interoperability and maintainability across teams.
- Support compliance and make external audits and certification possible.
- Capture industry best practices so teams need not reinvent them.

Processes

A defined process describes *who does what, when and how* - for example the requirements review process, code review process, test process and change control process.

- Make results repeatable and predictable rather than dependent on individuals.
- Allow quality checkpoints (entry and exit criteria) at every phase.
- Make it possible to measure and improve performance.
- Reduce errors, rework and misunderstandings.

Documentation

Documentation includes the SQA plan, requirements specification, design documents, test plans, test cases, test reports, defect reports, review records, audit reports and user manuals.

- Acts as the single source of truth and a communication tool between stakeholders.
- Provides **traceability** from requirements to design, code and tests.
- Serves as **evidence** of compliance for audits and customers.
- Supports maintenance, training and onboarding of new team members.
- Preserves knowledge and lessons learned for future projects.

Together, standards define *what* is expected, processes define *how* to achieve it, and documentation provides *proof and traceability* that it was done.

Q12. What is a Software Quality Assurance Plan (SQAP)? Explain its major components.

A **Software Quality Assurance Plan (SQAP)** is a formal document that describes how quality assurance will be carried out for a particular project: the standards, activities, responsibilities, resources, schedule and controls that will ensure the software meets its quality requirements. IEEE Std 730 provides a widely used structure for it.

Major Components of an SQAP

Component	Description
Purpose and scope	Why the plan exists, which software, project phases and activities it covers.
Reference documents	Standards, requirement specifications, contracts and other documents the plan relies on.
Management	Organisation structure, SQA roles, tasks and responsibilities, and reporting lines.
Documentation	List of documents to be produced and reviewed (SRS, design, test plan, user manual, etc.).
Standards, practices, conventions and metrics	Coding, design and documentation standards; quality metrics to be collected.
Reviews and audits	Types, schedule and participants of walkthroughs, inspections, technical reviews and audits.
Testing	Test strategy, levels of testing, tools, and entry/exit criteria (or reference to the test plan).
Problem reporting and corrective action	How defects and non-conformances are reported, tracked, prioritised and closed.
Tools, techniques and methodologies	Tools used for QA, such as defect trackers, static analysers and automation.
Code and media control	Configuration management, version control, backup, and protection of software media.
Supplier control	How quality of third-party / subcontracted software is ensured.
Records collection, maintenance and retention	Which quality records are kept, where, and for how long.
Training	Training needed so that the team can follow the processes and tools.
Risk management	Identification, assessment and mitigation of quality-related risks.
Schedule and glossary	Timeline of QA activities and definitions of terms used.

Q13. Explain the difference between Verification and Validation and discuss their role in software quality.

Verification asks *"Are we building the product right?"* - it checks that each work product conforms to its specification and to the previous phase. **Validation** asks *"Are we building the right product?"* - it checks that the finished product meets the user's actual needs and intended use.

Basis	Verification	Validation
Key question	Are we building the product right?	Are we building the right product?
Purpose	Ensure the product conforms to specifications and design.	Ensure the product meets user needs and expectations.
Nature	Static - no code execution needed.	Dynamic - the software is executed.
Activities	Reviews, walkthroughs, inspections, static analysis, desk-checking.	Functional, system and user acceptance testing; beta testing.

Basis	Verification	Validation
Applied to	Requirements, design, code, documents.	Working software / final product.
Timing	Early and throughout development.	After a working build is available.
Performed by	Developers, reviewers, QA team.	Testers and end users / customers.
Finds	Errors in documents and code logic before execution.	Mismatches between product behaviour and real needs.

Role in Software Quality

- Verification catches defects **early** (requirements, design, code), when they are cheapest to fix.
- Validation confirms that the delivered product is **fit for purpose** and acceptable to the customer.
- Together (as shown in the V-model) they make sure the product is both built correctly and is the correct product; one without the other leaves gaps - software may match its specification but still not satisfy users.

Q14. Explain how defects can affect the overall quality of software.

A **defect** (bug) is a flaw in a software work product that can cause it to fail to perform as required. Defects degrade almost every dimension of quality:

Quality Aspect	Effect of Defects
Functionality	Features give wrong results, are missing, or do not work at all (e.g. wrong interest calculation).
Reliability	Crashes, hangs, data corruption and unpredictable behaviour reduce trust in the system.
Performance	Memory leaks, inefficient queries and deadlocks slow down or stop the system.
Security	Defects such as injection flaws or weak validation can be exploited, leading to data breaches.
Usability	Confusing errors, broken navigation and inconsistent behaviour frustrate users.
Maintainability	Defect-prone, tangled code is hard to change; every fix risks creating new defects.
Compatibility / portability	Software misbehaves on certain devices, operating systems or browsers.

Business and Organisational Impact

- **Higher cost:** the later a defect is discovered, the more expensive it is to fix (rework, patching, redeployment).
- **Schedule delays** and overshooting budgets.
- **Customer dissatisfaction,** bad reviews, churn and loss of reputation.
- **Financial and legal consequences:** penalties, lawsuits and regulatory action.
- **Safety risks** in critical systems.

Real examples: in 1996 the first Ariane 5 rocket was destroyed shortly after launch because of a software data-conversion (overflow) error; in 2012 a faulty software deployment at Knight Capital

caused trading losses of roughly 440 million US dollars in under an hour. Both show how a single defect can have enormous consequences.

Q15-16. A banking application frequently crashes during online transactions. As a QA professional, explain what activities you would perform to improve the quality of the application.

Frequent crashes during online transactions are a **critical (severity-1)** problem: they put customers' money, data integrity and the bank's trust at risk. I would handle it in a structured way, combining immediate investigation, corrective testing, and long-term preventive QA.

Step 1: Triage and Information Gathering

- Classify the issue as critical and inform stakeholders; set up a dedicated task force (developers, DBAs, DevOps, QA).
- Collect crash reports, application and server logs, monitoring/APM data, transaction IDs, device/OS/browser information and customer complaints.
- Identify patterns: which transaction type, time of day, load level, user segment, device, network or release triggers the crash.

Step 2: Reproduce and Analyse the Root Cause

- Reproduce the crash in a test environment that mirrors production, using the same data volume and configuration.
- Investigate likely causes: unhandled exceptions, memory leaks, database deadlocks or connection-pool exhaustion, race conditions during concurrent transactions, session/timeout handling, slow or failing third-party/payment-gateway/core-banking calls, and recent code or configuration changes.
- Perform root cause analysis (5-Whys / fishbone) and document it.

Step 3: Strengthen Testing

Test Type	What I would verify
Functional / end-to-end	Complete transaction flows: login, add payee, fund transfer, bill payment, OTP/authentication, confirmation and statement update.
Negative and boundary	Invalid amounts, insufficient balance, expired sessions, wrong OTP, limits, special characters, double-click / repeated submit.
Integration / API	Interaction with core banking, payment gateway, SMS/OTP service and databases, including timeout and error responses.
Performance - load, stress, spike, endurance	Behaviour under normal, peak and sudden-surge traffic (e.g. salary day, festival sales) and over long durations (memory leaks).
Concurrency	Multiple simultaneous transactions on the same account; correct locking and no duplicate or lost debits/credits.
Data integrity and transaction (ACID)	Rollback on failure, no partial transactions, balances remain consistent after a crash.
Recovery and failover	Application and database restart, retry behaviour, and that interrupted transactions are safely resolved.
Security	Authentication, session management, encryption, vulnerability scanning and penetration testing.
Compatibility and network	Different devices, OS/browser versions, screen sizes and slow or interrupted networks.
Regression	Re-run the automated regression suite after each fix to ensure nothing else breaks.

Step 4: Defect Management and Fix Verification

- Log each defect with logs, steps to reproduce, severity/priority, and track it to closure.
- Verify every fix with retesting, plus regression and a repeat of the load test that exposed the crash.
- Do not release until exit criteria are met (e.g. zero open critical defects, stable under target load).

Step 5: Process-Level (QA) Improvements

- Introduce or enforce peer code reviews and static code analysis (to catch resource leaks and unhandled exceptions).
- Add performance and concurrency tests to the standard test strategy and the CI/CD pipeline; automate critical transaction regression tests.
- Improve requirement and design reviews, with non-functional requirements (response time, throughput, availability) clearly defined.
- Use a controlled release approach: staging environment parity, canary or phased rollout, and an agreed rollback plan.
- Set up production monitoring and alerts (error rate, response time, crash rate) and a clear incident-response process.
- Conduct a post-incident review (blameless retrospective), update checklists, test cases and standards, and track metrics such as crash-free sessions, defect leakage and mean time to recover.
- Ensure compliance with relevant banking and security regulations and standards (for example PCI DSS and the regulator's guidelines).

Outcome: the immediate crash cause is eliminated and verified, and the process improvements reduce the chance of similar failures in future releases, restoring customer trust.

Q17. Explain the importance of continuous improvement in Software Quality Assurance.

Continuous improvement means regularly reviewing and enhancing processes, practices and tools in small, ongoing steps instead of treating quality as a one-time achievement. It is built into models such as **PDCA (Plan-Do-Check-Act)**, **Kaizen**, Six Sigma, and the maturity levels of CMMI.

Why It Is Important

- **Technology and requirements keep changing,** so yesterday's process may not suit tomorrow's project.
- **Fewer defects over time:** root cause analysis and preventive actions stop repeat problems.
- **Lower cost of quality:** less rework, lower maintenance and support effort.
- **Faster delivery:** removing waste and automating repetitive work shortens cycle time.
- **Higher customer satisfaction:** customer feedback is fed back into the process.
- **Competitive advantage:** organisations that learn faster deliver better products.
- **Team motivation and learning:** retrospectives and shared ownership of quality build a quality culture.
- **Compliance:** standards such as ISO 9001 explicitly require continual improvement.

How It Is Practised

1. **Plan:** identify a problem or opportunity using metrics (e.g. high defect leakage) and set a target.
2. **Do:** pilot the change (e.g. add peer reviews or automated regression tests).
3. **Check:** measure the results and compare them with the target.
4. **Act:** standardise what worked, adjust or drop what did not, and start the next cycle.

Common inputs are defect metrics, audit findings, retrospectives, customer feedback and lessons learned.

Q18. What are the major challenges faced while implementing Software Quality Assurance in an organization?

Challenge	Explanation / Possible Remedy
Lack of management commitment	Quality seen as a cost, not an investment. Remedy: show ROI with data on defect cost and customer impact.
Resistance to change	Teams see QA as extra bureaucracy. Remedy: involve the team, communicate benefits, start small.
Time and budget pressure	Quality activities are cut when deadlines are tight. Remedy: build QA into the plan and estimates.
Shortage of skilled people	Lack of trained QA/testing professionals. Remedy: training, mentoring, certification.
Unclear or changing requirements	Hard to define quality criteria and tests. Remedy: requirement reviews, change control, early stakeholder involvement.
Difficulty measuring quality and ROI	Quality benefits are indirect and long-term. Remedy: define meaningful metrics and track trends.
Poor communication and collaboration	Silos between development, QA and operations. Remedy: cross-functional teams, DevOps practices.
Tool and automation problems	Wrong tools, high initial cost, flaky tests. Remedy: choose tools carefully, start with high-value automation.
Excessive documentation or rigid process	Process overhead reduces productivity. Remedy: tailor processes to project size and risk.
Agile / DevOps speed	Rapid releases leave little time for traditional QA. Remedy: shift-left, CI/CD test automation, continuous testing.
Legacy systems and distributed teams	Poorly documented code, time-zone and cultural differences. Remedy: standard processes, shared tools, clear documentation.
Inconsistent adherence to standards	Different teams follow different practices. Remedy: audits, templates, and a central QMS.

Q19. Explain the role of reviews, inspections, and audits in Software Quality Assurance.

Reviews, inspections and audits are **static quality activities**: they examine work products and processes without executing the software. They find problems early and give independent assurance of quality.

Reviews

A review is a systematic examination of a work product (requirements, design, code, test cases) by peers or stakeholders to find defects and improvement opportunities. Types include informal reviews, **walkthroughs** (author explains the work product to the team), **technical reviews** (experts evaluate suitability against standards and specifications) and management reviews (progress and status).

Inspections

An inspection is the most formal type of peer review (popularised by Michael Fagan). It follows a defined procedure with entry and exit criteria and specific roles: **moderator, author, reader, recorder and inspectors**. Typical stages are planning, overview, individual preparation, inspection meeting, rework and follow-up. Inspections are very effective at finding defects early and collect data for process improvement.

Audits

An audit is an independent, formal assessment of whether processes and products comply with standards, plans and contractual requirements. Audits can be internal or external (e.g. certification audits), and may be *process audits*, *product audits* or *configuration audits* (functional and physical). Findings are documented as non-conformances and tracked to closure through corrective actions.

Basis	Review	Inspection	Audit
Formality	Informal to semi-formal.	Highly formal.	Formal and independent.
Objective	Find defects and gather feedback.	Find defects using a strict procedure.	Check compliance with standards and plans.
Object examined	Work product.	Work product.	Process and/or product.
Conducted by	Peers / team.	Trained inspection team with defined roles.	Independent auditors / QA.
Output	Comments and action items.	Defect log and metrics.	Audit report with non-conformances.

Role in SQA

- Detect defects at the requirements, design and code stages when fixing is cheap.
- Verify compliance with standards and the SQA plan.
- Share knowledge and improve team understanding.
- Provide objective evidence and metrics for management and customers.
- Identify process weaknesses and trigger corrective and preventive actions.

Q20. Explain how QA activities are performed throughout the Software Development Life Cycle (SDLC).

QA is not a final step - it runs through **every phase** of the SDLC. The approach of building quality in from the beginning is often called **shift-left**.

SDLC Phase	QA Activities
Planning / Feasibility	Define quality objectives; prepare SQA plan and test strategy; identify risks; select standards, tools and metrics.
Requirements Analysis	Review requirements for completeness, clarity, consistency and testability; create requirements traceability matrix; plan acceptance criteria.
Design	Review architecture and design documents; check that non-functional needs (security, performance) are addressed; design test approach and test cases.

SDLC Phase	QA Activities
Implementation / Coding	Enforce coding standards; code reviews and static analysis; unit testing; configuration and version control.
Testing	Execute integration, system, regression, performance, security and usability tests; defect tracking and verification; evaluate against exit criteria; user acceptance testing support.
Deployment / Release	Release readiness review; configuration and build audits; smoke and sanity testing; deployment verification and rollback plan.
Maintenance	Regression testing for patches and enhancements; analyse production defects and customer feedback; root cause analysis and process improvement.

In Different Models

- **Waterfall / V-model:** each development phase has a matching verification or test phase (requirements - acceptance testing, design - system testing, etc.).
- **Agile:** QA works inside the sprint - reviewing user stories, defining acceptance criteria, testing continuously and joining retrospectives.
- **DevOps / CI-CD:** automated tests and quality gates run on every commit and release, and production monitoring feeds back into the process.

Q21. Explain with a suitable real-world example how effective Software Testing and Quality Assurance can improve customer satisfaction.

The following is an illustrative scenario. Consider an **online shopping application** preparing for a big festival sale, when traffic may be many times higher than normal.

Without Effective Testing and QA

- The app is released with minimal testing. When the sale starts, thousands of users log in at the same time.
- The server slows down, the payment page crashes, some customers are charged twice, and orders go missing.
- Customers cannot buy, complain on social media, uninstall the app and move to competitors; support is flooded; the company loses revenue and reputation.

With Effective Testing and QA

1. **Requirements review (QA):** expected load, response time and payment reliability are defined as measurable non-functional requirements.
2. **Standards and code reviews (QA):** reviews and static analysis catch coding errors and security flaws early.
3. **Functional and integration testing:** search, cart, coupons, payment gateway and order tracking are tested end-to-end, including failure cases such as payment timeouts.
4. **Performance and stress testing:** the system is tested at several times the expected peak load, and bottlenecks (database queries, caching) are fixed before the sale.

5 **Security and compatibility testing:** payment data protection and behaviour across devices, browsers and networks are verified.

6 **Regression automation and release checks:** every last-minute fix is retested; a rollback plan and monitoring are ready.

7 **Post-release monitoring and feedback (continuous improvement):** issues are tracked, analysed and used to improve the next release.

Result and Link to Customer Satisfaction

Customer Expectation	How Testing and QA Delivers It
The app is available and fast	Load and performance testing ensures it handles peak traffic.
My payment and personal data are safe	Security testing and secure coding standards protect data.
Orders and payments are correct	Functional, integration and data-integrity testing prevents double charges and lost orders.
It works on my phone and network	Compatibility and network testing across devices and conditions.
Problems are fixed quickly	Defect management, monitoring and continuous improvement shorten resolution time.

Customers get a smooth shopping experience, which leads to higher ratings, repeat purchases and recommendations, fewer support complaints and a stronger brand. Thus testing (finding defects) and QA (preventing them through good processes) work together to turn quality into customer satisfaction and business success.